Crime Management System - Project Structure Documentation

Version: 1.0.0
Build Tool: Maven
Java Version: 17
Framework: Spring Boot 3.2.0
UI Framework: Java Swing (FlatLaf)
Database: MySQL

Root Directory Files

File/Folder	Type	Description
`pom.xml`	Configuration	Maven Project Object Model file defining project dependencies, build configuration, and main class entry point
`README.md`	Documentation	Main project documentation with setup instructions, features, and usage guide
`DEMO.md`	Documentation	Demonstration and usage examples for the crime management system
`test_system.sh`	Script	Shell script for building and testing the application
`setup_database.sh`	Script	Shell script to initialize and configure MySQL database
`setup_database_fixed.sql`	SQL	Fixed database schema and sample data insertion script
`reset_database.sql`	SQL	SQL script to reset database to initial state
`setup_password.sh`	Script	Script to set up database user passwords
`.vscode/settings.json`	Configuration	Visual Studio Code workspace settings
`logs/`	Directory	Application log files directory (contains daily rotated log files)

Source Code Structure (`src/`)

Main Application (`src/main/`)

Java Source (`src/main/java/`)

Root Package (`com/`)

File	Description
`HashGenerator.java`	Utility class for generating BCrypt password hashes (standalone tool for admin password generation)

Core Package (`com/crimemanagement/`)

File	Description
`DesktopLauncher.java`	Main application entry point that initializes Swing UI, shows login dialog, and launches main window

Data Access Layer (`dao/`)

File	Description
`BaseDAO.java`	Interface defining common CRUD operations (create, read, update, delete, exists, count) for all DAO implementations
`CaseDAO.java`	Data Access Object for Case entities, handles database operations for crime cases including search and filtering
`CaseUpdateDAO.java`	Data Access Object for CaseUpdate entities, manages case update records and case history tracking
`CriminalDAO.java`	Data Access Object for Criminal entities, handles all database operations for criminal records
`EvidenceDAO.java`	Data Access Object for Evidence entities, manages evidence collection, storage, and retrieval operations
`FIRDAO.java`	Data Access Object for FIR (First Information Report) entities, handles FIR creation, updates, and queries
`OfficerDAO.java`	Data Access Object for Officer entities, manages police officer records and badge number lookups
`SuspectDAO.java`	Data Access Object for Suspect entities, handles suspect-case relationships and bail information
`UserAccountDAO.java`	Data Access Object for UserAccount entities, manages user authentication and role-based access control

Entity Layer (`entity/`)

File	Description
`Case.java`	Entity class representing a crime case with fields for case number, FIR reference, title, priority, status, and court information
`CaseUpdate.java`	Entity class representing case update records with update type, description, and officer information
`Criminal.java`	Entity class representing a criminal with personal details, alias, date of birth, address, phone, and criminal record
`Evidence.java`	Entity class representing evidence items with type, description, location, collection details, and storage information

File	Description
`FIR.java`	Entity class representing First Information Report with crime details, complainant information, incident date, and status
`Officer.java`	Entity class representing a police officer with badge number, name, rank, department, contact information
`Suspect.java`	Entity class representing a suspect linked to a case and criminal with role, arrest date, and bail information
`UserAccount.java`	Entity class representing user accounts with username, password hash, and role (ADMIN, OFFICER, COURT)

Service Layer (`service/`)

File	Description
`AdminService.java`	Service layer for administrative operations including user account management, creation, updates, and deletion
`AuthService.java`	Service layer for user authentication, password verification using BCrypt, and default admin account creation
`CourtService.java`	Service layer for court-related operations and case management from court perspective
`CriminalService.java`	Service layer for criminal management operations including CRUD operations and validation
`FIRService.java`	Service layer for FIR management including creation, updates, status changes, and officer assignment
`InvestigationService.java`	Service layer for case investigation management including case updates, status changes, and case CRUD operations
`OfficerService.java`	Service layer for officer management including CRUD operations, badge number validation, and officer lookup
`SearchService.java`	Service layer for comprehensive search functionality across all entities (FIRs, cases, criminals, evidence, officers)

User Interface Layer (`ui/`)

File	Description
`AdminPanel.java`	Swing panel for managing user accounts with add, edit, delete, and role assignment functionality (ADMIN only)
`CasesPanel.java`	Swing panel for managing crime cases with add, edit, delete, view operations and FIR validation (COURT can add/remove, OFFICER read-only)
`CriminalsPanel.java`	Swing panel for managing criminal records with full CRUD operations, phone validation, and age calculation
`FIRsPanel.java`	Swing panel for managing FIRs with add, edit, delete operations, date formatting, and phone number validation
`LoginDialog.java`	Swing login dialog with username/password authentication, role selection, and role-based access validation

File	Description
`MainWindow.java`	Main application window with tabbed interface, role-based menu visibility, and access information display
`OfficersPanel.java`	Swing panel for managing police officers with add, edit, delete operations and badge number management
`StatsPanel.java`	Swing panel displaying dashboard statistics with charts showing crime statistics and record counts
`SuspectsPanel.java`	Swing panel for managing suspects linked to cases and criminals with role, bail amount, and arrest date tracking
`UIUtils.java`	Utility class for common UI operations including table formatting, search functionality, and tooltip management

Utility Layer (`util/`)

File	Description
`DatabaseConnection.java`	Database connection utility using HikariCP connection pool with MySQL configuration and connection management

DTO Layer (`dto/`)

Directory	Description
`dto/`	Data Transfer Objects directory (currently empty after cleanup, previously contained web API request DTOs)

Repository Layer (`repository/`)

Directory	Description
`repository/`	Spring Data JPA repository interfaces directory (currently empty after cleanup, replaced by DAO pattern)

Resources (`src/main/resources/`)

File	Description
`application.properties`	Spring Boot application configuration including server settings and framework properties
`database.properties`	Database connection configuration file with MySQL connection parameters (URL, username, password)
`logback.xml`	Logback logging configuration defining log levels, appenders, and log file rotation settings

File	Description
`schema.sql`	Database schema definition SQL script creating all tables, indexes, and foreign key relationships
`sample_data.sql`	Sample data insertion SQL script with test records for officers, criminals, FIRs, and cases

Test Code (`src/test/`)

File	Description
`SmokeTest.java`	Basic smoke test to verify application can start and database connection works

---

Build Output (`target/`)

Directory/File	Description
`target/`	Maven build output directory containing compiled classes, resources, and generated JAR files
`target/classes/`	Compiled Java classes and resources copied from src/main
`target/generated-sources/`	Auto-generated source files (if any)
`target/maven-status/`	Maven build status and compilation metadata
`target/crime-management-system-1.0.0.jar`	Executable JAR file for the application

Project Architecture

Technology Stack

- **Language:** Java 17
- **Framework:** Spring Boot 3.2.0
- **UI:** Java Swing with FlatLaf (Modern Look & Feel)
- **Database:** MySQL with HikariCP connection pooling
- **Build Tool:** Maven
- **Logging:** SLF4J with Logback
- **Password Hashing:** BCrypt (jbcrypt)
- **Charts:** XChart library for statistics visualization

Architecture Pattern

- **Layered Architecture:**
  - **Presentation Layer:** Swing UI panels (`ui/`)

- **Service Layer:** Business logic (`service/`)
- **Data Access Layer:** DAO pattern (`dao/`)
- **Entity Layer:** Domain models (`entity/`)

Role-Based Access Control

- **ADMIN:** Full system access including user management
- **OFFICER:** Data management (add/edit/delete officers, criminals, FIRs, suspects) but cannot manage cases
- **COURT:** Case management (add/edit/delete cases) and read-only access to other data

Key Features

1. **User Authentication:** Secure login with role-based access
2. **Officer Management:** Complete CRUD for police officers
3. **Criminal Management:** Full criminal record management with personal details
4. **FIR Management:** First Information Report creation and tracking
5. **Case Management:** Crime case tracking with status, priority, and court information
6. **Suspect Management:** Linking suspects to cases and criminals
7. **Evidence Tracking:** Evidence collection and management (data model present)
8. **Search Functionality:** Comprehensive search across all entities
9. **Statistics Dashboard:** Visual charts showing system statistics
10. **Data Validation:** Phone number validation (10 digits), date validation, FIR validation

---

Database Schema

Core Tables

- `officers` - Police officer records
- `criminals` - Criminal records
- `firs` - First Information Reports
- `cases` - Crime cases
- `suspects` - Suspect-case relationships
- `evidence` - Evidence records
- `case_updates` - Case update history
- `users` - User accounts for authentication

Build and Run

Prerequisites

- Java 17 or higher
- Maven 3.6+
- MySQL 8.0+

Build Command

```
mvn clean compile
```

Run Command

```
mvn spring-boot:run
```

Create Executable JAR

```
mvn clean package
java -jar target/crime-management-system-1.0.0.jar
```

File Count Summary

- **Total Java Files:** 40+
- **Configuration Files:** 5
- **SQL Scripts:** 3
- **Shell Scripts:** 3
- **Documentation Files:** 2

Generated: $(date)
Project: Crime Management System
Version: 1.0.0